

Ajay Qt/QML TCP + UDP Assignment

Complete source code printout

Types used: unsigned char / unsigned int (per assignment struct spec)

Ajay & Smriti | Task period: 17 Sep - 23 Sep 2026

```
cmake_minimum_required(VERSION 3.16)
project(AjayQtQmlProject LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_AUTOMOC ON)
set(CMAKE_AUTORCC ON)

find_package(Qt6 REQUIRED COMPONENTS Core Network Quick Qml)

add_subdirectory(server)
add_subdirectory(client)
```

Ajay Qt/QML TCP + UDP Project

Qt 6 / C++ / QML implementation of the assigned task.

Assignment mapping

- TCP server: port 4001
- UDP broadcast: port 4002
- Client GUI: QML, runs on the main thread
- TCP sending: separate worker thread
- UDP receiving: separate worker thread
- Server TCP reception: separate thread
- Server UDP sending: separate thread
- Operations: Add, Update, Delete
- Client list changes only after a successful UDP ACK

Important protocol note

The assignment writes `float long;`. `long` is a C++ keyword, so the C++ member is named `longitude`.

The type and field order are otherwise kept the same.

The struct fields (`cmdID`, `UniqueID`, `respID`, `Ack`) use plain `unsigned char` and `unsigned int`, matching the assignment sheet's "Unsigned char" / "Unsigned int" wording exactly, rather than Qt's `quint8`/`quint32` aliases.

The assignment does not specify command IDs, so this project uses:

- 0x01 = Add
- 0x02 = Update
- 0x03 = Delete
- 0xFF = ACK success
- 0x00 = ACK failure

The wire structures are packed to keep the protocol sizes fixed.

Build

Open the top-level `CMakeLists.txt` in Qt Creator with Qt 6.

Build two targets:

- `AjayServer`
- `AjayClient`

Run the server first, then the client.

For two PCs, change `SERVER_ADDRESS` in `client/clientcontroller.cpp` from `127.0.0.1` to the server PC's IPv4 address.

UDP is broadcast to `255.255.255.255:4002`.

Project flow

Client QML -> ClientController -> TcpSender thread -> TCP 4001
-> ServerReceiver thread -> ServerSender thread -> UDP 4002
-> ClientUdpReceiver thread -> ClientController -> QML list

```
#ifndef PROTOCOL_H
#define PROTOCOL_H

#pragma pack(push, 1)

// TCP 4001 - Add / Update
struct AddDataCmd
{
    unsigned char cmdID;
    unsigned int UniqueID;
    float lat;
    float longitude; // assignment says "long"; long is a C++ keyword
    char comment[50];
};

// TCP 4001 - Delete
struct DeleteDataCmd
{
    unsigned char cmdID;
    unsigned int UniqueID;
};

// UDP 4002 - Add / Update response
struct AddDataResp
{
    unsigned char respID;
    unsigned char Ack; // 0x00 = Fail, 0xFF = Success
    unsigned int UniqueID;
    float lat;
    float longitude;
    char comment[50];
};

// UDP 4002 - Delete response
struct DeleteDataResp
{
    unsigned char respID;
    unsigned char Ack; // 0x00 = Fail, 0xFF = Success
    unsigned int UniqueID;
};

#pragma pack(pop)

// The packed sizes are part of the network protocol.
static_assert(sizeof(AddDataCmd) == 63, "AddDataCmd size must be 63");
static_assert(sizeof(DeleteDataCmd) == 5, "DeleteDataCmd size must be 5");
static_assert(sizeof(AddDataResp) == 64, "AddDataResp size must be 64");
static_assert(sizeof(DeleteDataResp) == 6, "DeleteDataResp size must be 6");

namespace Protocol{
    constexpr unsigned char CMD_ADD = 0x01;
    constexpr unsigned char CMD_UPDATE = 0x02;
    constexpr unsigned char CMD_DELETE = 0x03;
    constexpr unsigned char ACK_FAIL = 0x00;
    constexpr unsigned char ACK_SUCCESS = 0xFF;
}

#endif
```

```
set(CLIENT_SOURCES
    main.cpp
    clientcontroller.h
    clientcontroller.cpp
    tcpsender.h
    tcpsender.cpp
    udpreceiver.h
    udpreceiver.cpp
    qml.qrc
)

qt_add_executable(AjayClient
    ${CLIENT_SOURCES}
)

target_include_directories(AjayClient PRIVATE
    ${CMAKE_CURRENT_SOURCE_DIR}
    ${CMAKE_CURRENT_SOURCE_DIR}/../common
)

target_link_libraries(AjayClient PRIVATE
    Qt6::Core
    Qt6::Network
    Qt6::Quick
    Qt6::Qml
)
```

```

#include <QGuiApplication>
#include <QQmlApplicationEngine>
#include <QQmlContext>
#include <QThread>
#include "clientcontroller.h"
#include "tcpsender.h"
#include "udpreceiver.h"

int main(int argc, char *argv[])
{
    QGuiApplication app(argc, argv);

    ClientController controller;

    QThread tcpThread;
    QThread udpThread;
    TcpSender tcpSender;
    UdpReceiver udpReceiver;

    tcpSender.moveToThread(&tcpThread);
    udpReceiver.moveToThread(&udpThread);

    QObject::connect(&tcpThread, &QThread::started,
                    &tcpSender, &TcpSender::start);
    QObject::connect(&udpThread, &QThread::started,
                    &udpReceiver, &UdpReceiver::start);

    QObject::connect(&controller, &ClientController::sendAddRequest,
                    &tcpSender, &TcpSender::sendAdd,
                    Qt::QueuedConnection);
    QObject::connect(&controller, &ClientController::sendUpdateRequest,
                    &tcpSender, &TcpSender::sendUpdate,
                    Qt::QueuedConnection);
    QObject::connect(&controller, &ClientController::sendDeleteRequest,
                    &tcpSender, &TcpSender::sendDelete,
                    Qt::QueuedConnection);

    QObject::connect(&udpReceiver, &UdpReceiver::addResponseReceived,
                    &controller, &ClientController::onAddResponse,
                    Qt::QueuedConnection);
    QObject::connect(&udpReceiver, &UdpReceiver::deleteResponseReceived,
                    &controller, &ClientController::onDeleteResponse,
                    Qt::QueuedConnection);

    QObject::connect(&app, &QGuiApplication::aboutToQuit, [&]()
    {
        QMetaObject::invokeMethod(&tcpSender, "stop",
                                   Qt::BlockingQueuedConnection);
        QMetaObject::invokeMethod(&udpReceiver, "stop",
                                   Qt::BlockingQueuedConnection);

        tcpThread.quit();
        udpThread.quit();
        tcpThread.wait();
        udpThread.wait();
    });

    tcpThread.start();
    udpThread.start();

    QQmlApplicationEngine engine;
    engine.rootContext()->setContextProperty("clientController", &controller);

    const QUrl url(QStringLiteral("qrc:/AjayClient/qml/Main.qml"));
    QObject::connect(
        &engine,

```

```
        &QQmlApplicationEngine::objectCreationFailed,  
        &app,  
        []() { QCoreApplication::exit(-1); },  
        Qt::QueuedConnection);  
engine.load(url);  
  
if (engine.rootObjects().isEmpty())  
    return -1;  
  
return app.exec();  
}
```

```
#ifndef CLIENTCONTROLLER_H
#define CLIENTCONTROLLER_H

#include <QObject>
#include <QVariantList>
#include <QVariantMap>

class ClientController : public QObject
{
    Q_OBJECT
public:
    explicit ClientController(QObject *parent = nullptr);

    Q_PROPERTY(QVariantList records READ records NOTIFY recordsChanged)
    QVariantList records() const;

    Q_INVOKABLE void addRecord(int uniqueID, double lat,
                               double longitude, const QString &comment);
    Q_INVOKABLE void updateRecord(int uniqueID, double lat,
                                   double longitude, const QString &comment);
    Q_INVOKABLE void deleteRecord(int uniqueID);

signals:
    void recordsChanged();
    void sendAddRequest(int uniqueID, double lat,
                        double longitude, const QString &comment);
    void sendUpdateRequest(int uniqueID, double lat,
                           double longitude, const QString &comment);
    void sendDeleteRequest(int uniqueID);

public slots:
    void onAddResponse(unsigned char respID, unsigned char ack,
                       unsigned int uniqueID, float lat,
                       float longitude, const QString &comment);
    void onDeleteResponse(unsigned char respID, unsigned char ack,
                          unsigned int uniqueID);

private:
    void addOrUpdateLocalRecord(unsigned int uniqueID, float lat,
                                float longitude, const QString &comment);

    QVariantList m_records;
};

#endif
```

```
#include "clientcontroller.h"
#include <QDebug>
#include "protocol.h"

ClientController::ClientController(QObject *parent)
    : QObject(parent)
{
}

QVariantList ClientController::records() const
{
    return m_records;
}

void ClientController::addRecord(int uniqueID, double lat,
                                double longitude, const QString &comment)
{
    emit sendAddRequest(uniqueID, lat, longitude, comment);
}

void ClientController::updateRecord(int uniqueID, double lat,
                                    double longitude, const QString &comment)
{
    emit sendUpdateRequest(uniqueID, lat, longitude, comment);
}

void ClientController::deleteRecord(int uniqueID)
{
    emit sendDeleteRequest(uniqueID);
}

void ClientController::onAddResponse(unsigned char respID, unsigned char ack,
                                     unsigned int uniqueID, float lat,
                                     float longitude,
                                     const QString &comment)
{
    {
        Q_UNUSED(respID);
        if (ack != Protocol::ACK_SUCCESS)
        {
            qDebug() << "Server rejected add/update:" << uniqueID;
            return;
        }
        addOrUpdateLocalRecord(uniqueID, lat, longitude, comment);
    }
}

void ClientController::onDeleteResponse(unsigned char respID, unsigned char ack,
                                       unsigned int uniqueID)
{
    {
        Q_UNUSED(respID);
        if (ack != Protocol::ACK_SUCCESS)
        {
            qDebug() << "Server rejected delete:" << uniqueID;
            return;
        }
        for (int i = 0; i < m_records.size(); ++i)
        {
            QVariantMap item = m_records[i].toMap();
            if (item["uniqueID"].toUInt() == uniqueID)
            {
                m_records.removeAt(i);
                emit recordsChanged();
                return;
            }
        }
    }
}
```

```
void ClientController::addOrUpdateLocalRecord(unsigned int uniqueID,
                                             float lat,
                                             float longitude,
                                             const QString &comment)
{
    for (int i = 0; i < m_records.size(); ++i)
    {
        QVariantMap item = m_records[i].toMap();
        if (item["uniqueID"].toUInt() == uniqueID)
        {
            item["lat"] = lat;
            item["longitude"] = longitude;
            item["comment"] = comment;
            m_records[i] = item;
            emit recordsChanged();
            return;
        }
    }

    QVariantMap item;
    item["uniqueID"] = uniqueID;
    item["lat"] = lat;
    item["longitude"] = longitude;
    item["comment"] = comment;
    m_records.append(item);
    emit recordsChanged();
}
```

```
#ifndef TCPSENDER_H
#define TCPSENDER_H

#include <QObject>
#include <QTcpSocket>

class TcpSender : public QObject
{
    Q_OBJECT
public:
    explicit TcpSender(QObject *parent = nullptr);

public slots:
    void start();
    void stop();
    void sendAdd(int uniqueID, double lat,
                double longitude, const QString &comment);
    void sendUpdate(int uniqueID, double lat,
                   double longitude, const QString &comment);
    void sendDelete(int uniqueID);

private:
    bool ensureConnected();

private:
    QTcpSocket *m_socket;
};

#endif
```

```

#include "tcpsender.h"
#include <QDebug>
#include <cstring>
#include "protocol.h"

static const QString SERVER_ADDRESS = QStringLiteral("127.0.0.1");

TcpSender::TcpSender(QObject *parent)
    : QObject(parent),
      m_socket(new QTcpSocket(this))
{
}

void TcpSender::start()
{
    qDebug() << "TCP sender thread ready.";
}

void TcpSender::stop()
{
    if (m_socket->state() != QAbstractSocket::UnconnectedState)
        m_socket->disconnectFromHost();
}

bool TcpSender::ensureConnected()
{
    if (m_socket->state() == QAbstractSocket::ConnectedState)
        return true;

    m_socket->connectToHost(SERVER_ADDRESS, 4001);
    if (!m_socket->waitForConnected(1000))
    {
        qDebug() << "Cannot connect to server:"
                  << m_socket->errorString();
        return false;
    }
    return true;
}

void TcpSender::sendAdd(int uniqueID, double lat,
                        double longitude, const QString &comment)
{
    if (!ensureConnected())
        return;

    AddDataCmd cmd{};
    cmd.cmdID = Protocol::CMD_ADD;
    cmd.UniqueID = static_cast<unsigned int>(uniqueID);
    cmd.lat = static_cast<float>(lat);
    cmd.longitude = static_cast<float>(longitude);

    QByteArray text = comment.toUtf8();
    std::strncpy(cmd.comment, text.constData(),
                  sizeof(cmd.comment) - 1);
    cmd.comment[sizeof(cmd.comment) - 1] = '\0';

    m_socket->write(reinterpret_cast<const char *>(&cmd),
                    sizeof(cmd));
    m_socket->flush();
}

void TcpSender::sendUpdate(int uniqueID, double lat,
                           double longitude, const QString &comment)
{
    if (!ensureConnected())

```

```
    return;

    AddDataCmd cmd{};
    cmd.cmdID = Protocol::CMD_UPDATE;
    cmd.UniqueID = static_cast<unsigned int>(uniqueID);
    cmd.lat = static_cast<float>(lat);
    cmd.longitude = static_cast<float>(longitude);

    QByteArray text = comment.toUtf8();
    std::strncpy(cmd.comment, text.constData(),
                sizeof(cmd.comment) - 1);
    cmd.comment[sizeof(cmd.comment) - 1] = '\0';

    m_socket->write(reinterpret_cast<const char *>(&cmd),
                    sizeof(cmd));
    m_socket->flush();
}

void TcpSender::sendDelete(int uniqueID)
{
    if (!ensureConnected())
        return;

    DeleteDataCmd cmd{};
    cmd.cmdID = Protocol::CMD_DELETE;
    cmd.UniqueID = static_cast<unsigned int>(uniqueID);

    m_socket->write(reinterpret_cast<const char *>(&cmd),
                    sizeof(cmd));
    m_socket->flush();
}
```

```
#ifndef UDPRECEIVER_H
#define UDPRECEIVER_H

#include <QObject>
#include <QUdpSocket>

class UdpReceiver : public QObject
{
    Q_OBJECT
public:
    explicit UdpReceiver(QObject *parent = nullptr);

public slots:
    void start();
    void stop();

signals:
    void addResponseReceived(unsigned char respID, unsigned char ack,
                             unsigned int uniqueID, float lat,
                             float longitude,
                             const QString &comment);
    void deleteResponseReceived(unsigned char respID, unsigned char ack,
                                unsigned int uniqueID);

private slots:
    void readPendingDatagrams();

private:
    QUdpSocket *m_socket;
};

#endif
```

```

#include "udpreceiver.h"
#include <QDebug>
#include <cstring>
#include "protocol.h"

UdpReceiver::UdpReceiver(QObject *parent)
    : QObject(parent),
      m_socket(new QUdpSocket(this))
{
    connect(m_socket, &QUdpSocket::readyRead,
            this, &UdpReceiver::readPendingDatagrams);
}

void UdpReceiver::start()
{
    bool ok = m_socket->bind(QHostAddress::AnyIPv4,
                             4002,
                             QUdpSocket::ShareAddress |
                             QUdpSocket::ReuseAddressHint);

    if (!ok)
    {
        qDebug() << "UDP bind failed:" << m_socket->errorString();
        return;
    }
    qDebug() << "UDP receiver thread listening on 4002";
}

void UdpReceiver::stop()
{
    m_socket->close();
}

void UdpReceiver::readPendingDatagrams()
{
    while (m_socket->hasPendingDatagrams())
    {
        QByteArray datagram;
        datagram.resize(static_cast<int>(m_socket->pendingDatagramSize()));
        m_socket->readDatagram(datagram.data(), datagram.size());

        if (datagram.isEmpty())
            continue;

        unsigned char responseID = static_cast<unsigned char>(datagram.at(0));

        if (responseID == Protocol::CMD_ADD ||
            responseID == Protocol::CMD_UPDATE)
        {
            if (datagram.size() != static_cast<int>(sizeof(AddDataResp)))
                continue;

            AddDataResp response{};
            std::memcpy(&response, datagram.constData(), sizeof(response));

            QString comment = QString::fromUtf8(response.comment);
            emit addResponseReceived(response.respID,
                                     response.Ack,
                                     response.UniqueID,
                                     response.lat,
                                     response.longitude,
                                     comment);
        }
        else if (responseID == Protocol::CMD_DELETE)
        {
            if (datagram.size() != static_cast<int>(sizeof(DeleteDataResp)))

```

```
        continue;

        DeleteDataResp response{};
        std::memcpy(&response, datagram.constData(), sizeof(response));

        emit deleteResponseReceived(response.resplD, response.Ack,
                                     response.UniqueID);
    }
}
```

```

import QtQuick
import QtQuick.Controls
import QtQuick.Layouts

ApplicationWindow {
    id: window
    width: 800
    height: 600
    visible: true
    title: "Ajay - TCP / UDP Client"

    property bool formOpen: false
    property bool editMode: false
    property int editId: -1
    property string statusText: "Ready"

    ColumnLayout {
        anchors.fill: parent
        anchors.margins: 20
        spacing: 12

        Label {
            text: "Data List"
            font.pixelSize: 26
            font.bold: true
        }

        Label {
            text: statusText
            color: "#555555"
        }

        ListView {
            id: recordList
            Layout.fillWidth: true
            Layout.fillHeight: true
            model: clientController.records
            clip: true

            delegate: Rectangle {
                width: recordList.width
                height: 72
                radius: 6
                border.width: 1
                border.color: "#cccccc"

                RowLayout {
                    anchors.fill: parent
                    anchors.margins: 10

                    ColumnLayout {
                        Layout.fillWidth: true
                        Label {
                            text: "ID: " + modelData.uniqueID
                            font.bold: true
                        }
                        Label {
                            text: "Lat: " + Number(modelData.lat).toFixed(4)
                                + " Long: "
                                + Number(modelData.longitude).toFixed(4)
                        }
                        Label {
                            text: modelData.comment
                            elide: Text.ElideRight
                            Layout.fillWidth: true
                        }
                    }
                }
            }
        }
    }
}

```

```

    }
}

Button {
    text: "Update"
    onClicked: {
        window.editMode = true
        window.editId = modelData.uniqueID
        recordForm.uniqueID = modelData.uniqueID
        recordForm.lat = modelData.lat
        recordForm.longitude = modelData.longitude
        recordForm.comment = modelData.comment
        window.formOpen = true
    }
}
Button {
    text: "Delete"
    onClicked: {
        clientController.deleteRecord(modelData.uniqueID)
        window.statusText =
            "Delete request sent. Waiting for ACK..."
    }
}
}
}

Label {
    anchors.centerIn: parent
    visible: clientController.records.length === 0
    text: "No records yet. Click Add."
    color: "#777777"
}
}

RowLayout {
    Layout.fillWidth: true
    Button {
        text: "Add"
        onClicked: {
            window.editMode = false
            window.editId = -1
            recordForm.uniqueID = 0
            recordForm.lat = 0
            recordForm.longitude = 0
            recordForm.comment = ""
            window.formOpen = true
        }
    }
}
}

RecordForm {
    id: recordForm
    visible: window.formOpen
    anchors.centerIn: parent
    editMode: window.editMode

    onApplyClicked: {
        if (editMode) {
            clientController.updateRecord(
                uniqueID, lat, longitude, comment)
            window.statusText =
                "Update request sent. Waiting for ACK..."
        } else {
            clientController.addRecord(

```

```
        uniqueID, lat, longitude, comment)
    window.statusText =
        "Add request sent. Waiting for ACK..."
    }
    window.formOpen = false
}
onCancelClicked: {
    window.formOpen = false
}
}

Connections {
    target: clientController
    function onRecordsChanged() {
        window.statusText = "Server ACK received. List updated."
    }
}
}
```

```
import QtQuick
import QtQuick.Controls
import QtQuick.Layouts

Rectangle {
    id: form
    width: 430
    height: 370
    radius: 10
    border.width: 1
    border.color: "#aaaaaa"
    color: "white"

    property bool editMode: false
    property int uniqueID: 0
    property real lat: 0
    property real longitude: 0
    property string comment: ""

    signal applyClicked()
    signal cancelClicked()

    ColumnLayout {
        anchors.fill: parent
        anchors.margins: 20
        spacing: 12

        Label {
            text: editMode ? "Update Record" : "Add Record"
            font.pixelSize: 22
            font.bold: true
        }

        TextField {
            id: idField
            Layout.fillWidth: true
            placeholderText: "Unique ID"
            text: String(form.uniqueID)
            inputMethodHints: Qt.ImhDigitsOnly
            onTextChanged: {
                if (activeFocus)
                    form.uniqueID = Number(text)
            }
        }

        TextField {
            id: latField
            Layout.fillWidth: true
            placeholderText: "Latitude"
            text: String(form.lat)
            onTextChanged: {
                if (activeFocus)
                    form.lat = Number(text)
            }
        }

        TextField {
            id: longField
            Layout.fillWidth: true
            placeholderText: "Longitude"
            text: String(form.longitude)
            onTextChanged: {
                if (activeFocus)
                    form.longitude = Number(text)
            }
        }
    }
}
```

```

    }

    TextField {
        id: commentField
        Layout.fillWidth: true
        placeholderText: "Comment (maximum 49 characters)"
        text: form.comment
        maximumLength: 49
        onTextChanged: {
            form.comment = text
        }
    }
}

RowLayout {
    Layout.fillWidth: true
    Button {
        text: "Apply"
        Layout.fillWidth: true
        onClicked: {
            form.uniqueID = Number(idField.text)
            form.lat = Number(latField.text)
            form.longitude = Number(longField.text)
            form.comment = commentField.text
            form.applyClicked()
        }
    }
    Button {
        text: "Cancel"
        Layout.fillWidth: true
        onClicked: form.cancelClicked()
    }
}

// When a form is opened, copy the current properties into the fields.
onUniqueIDChanged: {
    if (!idField.activeFocus)
        idField.text = String(uniqueID)
}
onLatChanged: {
    if (!latField.activeFocus) latField.text = String(lat)
}
onLongitudeChanged: {
    if (!longField.activeFocus)
        longField.text = String(longitude)
}
onCommentChanged: {
    if (!commentField.activeFocus)
        commentField.text = comment
}
}

```

```
<!DOCTYPE RCC><RCC version="1.0">  
<qresource prefix="/AjayClient/qml">  
  <file>Main.qml</file>  
  <file>RecordForm.qml</file>  
</qresource>  
</RCC>
```

```
set(SERVER_SOURCES
    main.cpp
    serverreceiver.h
    serverreceiver.cpp
    serversender.h
    serversender.cpp
)

add_executable(AjayServer ${SERVER_SOURCES})

target_include_directories(AjayServer PRIVATE
    ${CMAKE_CURRENT_SOURCE_DIR}
    ${CMAKE_CURRENT_SOURCE_DIR}/../common
)

target_link_libraries(AjayServer PRIVATE
    Qt6::Core
    Qt6::Network
)
```

```
#include <QCoreApplication>
#include <QThread>
#include <QDebug>
#include "serverreceiver.h"
#include "serversender.h"

int main(int argc, char *argv[])
{
    QCoreApplication app(argc, argv);

    QThread receiverThread;
    QThread senderThread;

    ServerReceiver receiver;
    ServerSender sender;

    receiver.moveToThread(&receiverThread);
    sender.moveToThread(&senderThread);

    QObject::connect(&receiver, &ServerReceiver::commandReceived,
                    &sender, &ServerSender::processCommand,
                    Qt::QueuedConnection);

    QObject::connect(&receiverThread, &QThread::started,
                    &receiver, &ServerReceiver::start);
    QObject::connect(&senderThread, &QThread::started,
                    &sender, &ServerSender::start);

    QObject::connect(&app, &QCoreApplication::aboutToQuit, [&]()
    {
        QMetaObject::invokeMethod(&receiver, "stop",
                                   Qt::BlockingQueuedConnection);
        QMetaObject::invokeMethod(&sender, "stop",
                                   Qt::BlockingQueuedConnection);
        receiverThread.quit();
        senderThread.quit();
        receiverThread.wait();
        senderThread.wait();
    });

    receiverThread.start();
    senderThread.start();

    qDebug() << "Ajay Server started.";
    qDebug() << "TCP port: 4001";
    qDebug() << "UDP broadcast port: 4002";

    return app.exec();
}
```

```
#ifndef SERVERRECEIVER_H
#define SERVERRECEIVER_H

#include <QObject>
#include <QTcpServer>
#include <QTcpSocket>
#include <QHash>
#include <QByteArray>

class ServerReceiver : public QObject
{
    Q_OBJECT
public:
    explicit ServerReceiver(QObject *parent = nullptr);

public slots:
    void start();
    void stop();

signals:
    void commandReceived(const QByteArray &data);

private slots:
    void onNewConnection();
    void onReadyRead();
    void onDisconnected();

private:
    QTcpServer *m_server;
    QHash<QTcpSocket *, QByteArray> m_buffers;
};

#endif
```

```
#include "serverreceiver.h"
#include <QDebug>
#include "protocol.h"

ServerReceiver::ServerReceiver(QObject *parent)
    : QObject(parent),
      m_server(new QTcpServer(this))
{
    connect(m_server, &QTcpServer::newConnection,
           this, &ServerReceiver::onNewConnection);
}

void ServerReceiver::start()
{
    if (!m_server->listen(QHostAddress::Any, 4001))
    {
        qDebug() << "TCP listen failed:" << m_server->errorString();
        return;
    }
    qDebug() << "TCP receiver thread listening on 4001";
}

void ServerReceiver::stop()
{
    for (QTcpSocket *socket : m_buffers.keys())
    {
        socket->disconnectFromHost();
    }
    m_buffers.clear();

    if (m_server->isListening())
    {
        m_server->close();
    }
}

void ServerReceiver::onNewConnection()
{
    while (m_server->hasPendingConnections())
    {
        QTcpSocket *socket = m_server->nextPendingConnection();
        m_buffers[socket] = QByteArray();

        connect(socket, &QTcpSocket::readyRead,
               this, &ServerReceiver::onReadyRead);
        connect(socket, &QTcpSocket::disconnected,
               this, &ServerReceiver::onDisconnected);

        qDebug() << "Client connected:" << socket->peerAddress().toString();
    }
}

void ServerReceiver::onReadyRead()
{
    QTcpSocket *socket = qobject_cast<QTcpSocket *>(sender());
    if (!socket)
        return;

    QByteArray &buffer = m_buffers[socket];
    buffer.append(socket->readAll());

    // A packet can arrive in parts, so keep the bytes until
    // one complete command is available.
    while (true)
    {

```

```
    if (buffer.isEmpty())
        return;

    unsigned char cmdID = static_cast<unsigned char>(buffer.at(0));
    int packetSize = 0;

    if (cmdID == Protocol::CMD_ADD || cmdID == Protocol::CMD_UPDATE)
        packetSize = sizeof(AddDataCmd);
    else if (cmdID == Protocol::CMD_DELETE)
        packetSize = sizeof(DeleteDataCmd);
    else
    {
        qDebug() << "Unknown command ID:" << cmdID;
        buffer.clear();
        return;
    }

    if (buffer.size() < packetSize)
        return;

    QByteArray packet = buffer.left(packetSize);
    buffer.remove(0, packetSize);
    emit commandReceived(packet);
}

void ServerReceiver::onDisconnected()
{
    QTcpSocket *socket = qobject_cast<QTcpSocket *>(sender());
    if (!socket)
        return;

    qDebug() << "Client disconnected.";
    m_buffers.remove(socket);
    socket->deleteLater();
}
```

```
#ifndef SERVERSENDER_H
#define SERVERSENDER_H

#include <QObject>
#include <QUdpSocket>
#include <QHash>

class ServerSender : public QObject
{
    Q_OBJECT
public:
    explicit ServerSender(QObject *parent = nullptr);

public slots:
    void start();
    void stop();
    void processCommand(const QByteArray &data);

private:
    void sendAddResponse(unsigned char respID, unsigned char ack,
                        unsigned int uniqueID, float lat,
                        float longitude, const char *comment);
    void sendDeleteResponse(unsigned char respID, unsigned char ack,
                        unsigned int uniqueID);

private:
    QUdpSocket *m_udpSocket;
    // Server keeps the current records so Update/Delete can be checked.
    QHash<unsigned int, QByteArray> m_records;
};

#endif
```

```

#include "serversender.h"
#include <QDebug>
#include <cstring>
#include "protocol.h"

ServerSender::ServerSender(QObject *parent)
    : QObject(parent),
      m_udpSocket(new QUdpSocket(this))
{
}

void ServerSender::start()
{
    qDebug() << "UDP sender thread ready.";
}

void ServerSender::stop()
{
    if (m_udpSocket)
        m_udpSocket->close();
}

void ServerSender::processCommand(const QByteArray &data)
{
    if (data.isEmpty())
        return;

    unsigned char cmdID = static_cast<unsigned char>(data.at(0));

    if (cmdID == Protocol::CMD_ADD || cmdID == Protocol::CMD_UPDATE)
    {
        if (data.size() != static_cast<int>(sizeof(AddDataCmd)))
            return;

        AddDataCmd cmd{};
        std::memcpy(&cmd, data.constData(), sizeof(cmd));

        bool exists = m_records.contains(cmd.UniqueID);
        bool allowed = false;

        if (cmdID == Protocol::CMD_ADD)
            allowed = !exists;
        else
            allowed = exists;

        if (allowed)
        {
            m_records.insert(cmd.UniqueID, data);
            sendAddResponse(cmdID, Protocol::ACK_SUCCESS,
                           cmd.UniqueID, cmd.lat, cmd.longitude, cmd.comment);
            qDebug() << (cmdID == Protocol::CMD_ADD ? "ADD" : "UPDATE")
                       << "success:" << cmd.UniqueID;
        }
        else
        {
            sendAddResponse(cmdID, Protocol::ACK_FAIL,
                           cmd.UniqueID, cmd.lat,
                           cmd.longitude, cmd.comment);
            qDebug() << (cmdID == Protocol::CMD_ADD ? "ADD" : "UPDATE")
                       << "failed:" << cmd.UniqueID;
        }
    }
    else if (cmdID == Protocol::CMD_DELETE)
    {
        if (data.size() != static_cast<int>(sizeof(DeleteDataCmd)))

```

```

        return;

        DeleteDataCmd cmd{};
        std::memcpy(&cmd, data.constData(), sizeof(cmd));

        bool exists = m_records.contains(cmd.UniqueID);
        if (exists)
        {
            m_records.remove(cmd.UniqueID);
            sendDeleteResponse(Protocol::CMD_DELETE,
                               Protocol::ACK_SUCCESS,
                               cmd.UniqueID);
            qDebug() << "DELETE success:" << cmd.UniqueID;
        }
        else
        {
            sendDeleteResponse(Protocol::CMD_DELETE,
                               Protocol::ACK_FAIL,
                               cmd.UniqueID);
            qDebug() << "DELETE failed:" << cmd.UniqueID;
        }
    }
}

void ServerSender::sendAddResponse(unsigned char respID, unsigned char ack,
                                   unsigned int uniqueID, float lat,
                                   float longitude, const char *comment)
{
    AddDataResp response{};
    response.respID = respID;
    response.Ack = ack;
    response.UniqueID = uniqueID;
    response.lat = lat;
    response.longitude = longitude;
    std::strncpy(response.comment, comment, sizeof(response.comment) - 1);
    response.comment[sizeof(response.comment) - 1] = '\0';

    QByteArray packet(reinterpret_cast<const char *>(&response),
                       sizeof(response));
    m_udpSocket->writeDatagram(packet,
                               QHostAddress::Broadcast,
                               4002);
}

void ServerSender::sendDeleteResponse(unsigned char respID, unsigned char ack,
                                      unsigned int uniqueID)
{
    DeleteDataResp response{};
    response.respID = respID;
    response.Ack = ack;
    response.UniqueID = uniqueID;

    QByteArray packet(reinterpret_cast<const char *>(&response),
                       sizeof(response));
    m_udpSocket->writeDatagram(packet,
                               QHostAddress::Broadcast,
                               4002);
}

```